VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



COMPUTER ARCHITECTURE (LAB) (CO2008)

Assignment - Semester 242

Group 5 - Class CC02

Five in a Row in Assembly

Advisor(s): Nguyễn Thiên Ân
Student(s): Nguyễn Minh Khôi - 2252376
Huỳnh Đào Đông Quân - 2053367
Trần Đình Duy Khương - 2352638

HO CHI MINH CITY, APRIL 2025

Contents

1 Introduction **4**

2 Board Management **4**

3 Input Handling **5**

4 Move marking **5**

5 Win Detection **5**

6 Tie Detection **6**

7 Outputting Results **6**

8 Conclusion **7**

Opening Statement

Dear **Mr. Nguyen Thien An**,

We hope this message finds you well. We would like to express our sincere gratitude for the valuable guidance and support you have provided us during the Computer Architecture Lab course.

In particular, working on the Five-in-a-Row game assignment in MIPS Assembly has greatly enhanced our practical understanding of low-level programming and computer architecture principles.

Through this project, we gained hands-on experience in:

- Implementing data structures and algorithms at the assembly level
- Managing memory directly and handling user inputs without the support of high-level abstractions
- Applying logical and systematic thinking to solve complex problems like win detection, input parsing, and file I/O

This assignment not only helped reinforce key concepts from the course but also laid an important foundation for our future studies in areas such as operating systems, systems programming, and computer networks.

As third-year students, while we have tried our best to apply our knowledge, we acknowledge that there may be shortcomings in our implementation. We sincerely look forward to your feedback and evaluation so that we can continue improving.

Once again, we would like to extend our heartfelt thanks to the Faculty of Applied Science and the Department of Computer Science for creating such a meaningful opportunity for learning and growth.

Best regards,
Group 15 of class CC02

1 Introduction

The purpose of this project is to develop a two-player game of "Caro", otherwise known as Five in a Row using MIPS assembly language.
The gameplay flow is as follows:

- The game is played on a 15x15 board, via a 255 bytes array. Each element will be a character, with the size of 1 byte.
- 2 players (each representing 'X' and 'O', respectively) will enter their coordinates in the format of x,y with comma as the seperator, to determine the position of their piece on the board
- The game ends when one of two conditions are satisfied:
  - A player gets 5 pieces in a line (either horizontally, vertically, or diagonally), resulting in a win
  - The board is filled but none of the players found the win condition, resulting in a tie.
- Repeat the first step until all of the squares on the board are played

2 Board Management

Our board is implemented as a one-dimensional array containing 225 cells, corresponding to the 15x15 grid.

Each cells stores a character:

- '.': represents an empty space
- 'X': represents Player 1
- 'O': represents Player 2

To access a specified cell based on the inputted coordinates (x,y), the index is calculated using the following formula:

$$index = 15 * x + y$$

where x and y are the coordinates from the player's input.

This simplifies the storage and retrieval of board data while handling a 2-dimensional board in linear memory.



3 Input Handling

Players are prompted to input their move coordinates in the format "x,y", with the comma (',') acting as the separator. The program reads the input as a string, then parses it character by character:

- Characters before the comma are interpreted as the x-coordinate.
- Characters after the comma are interpreted as the y-coordinate.

The program is also implemented with an input validation, making sure that:

- Both x and y are integers within the range of [0,14].
- The input format strictly matches expectations (only digits and a single comma).

If the input is invalid (wrong format, non-numeric characters, or out-of-range numbers), the program displays an error message and requests the player to input again.

4 Move marking

After validation the input:

- The program checks if the chosen board cell is empty ('.').
- If the cell is occupied, it notifies the player and asks for a new input without losing their turn.
- If the cell is empty, it marks:
  - 'X' on Player 1's turn.
  - 'O' on Player 2's turn.

This ensures that players only move into valid, available spaces.

5 Win Detection

After each move, the program checks if the current player has achieved five consecutive sumbols in any direction.

The checking process covers four directions:

- **Horizontal** (left and right from the current cell)
- **Vertical** (up and down)
- **Diagonal** (top-left to bottom-right)
- **Anti-Diagonal** (top-right to bottom-left)

For each direction:

- The program counts consecutive identical symbols from the current move outward.
- It combines the counts from both directions (e.g., left and right) and includes the currect cell.
- If the total reaches five or more, the player wins immediately.

This systematic checking ensures accurate win detection with minimal redundant checks.

6 Tie Detection

If no player has won yet, the program checks whether the board is full. This is done by scanning every cell to detect if any '.' (empty space) remains.

If no empty spaces found and neither player has won, the game is declared a tie.

This method guarantees that ties are only recognized when the entire board has been exhausted without any five-in-a-row sequence.

7 Outputting Results

Upon game termination, the final state of the board and the result message are saved into a text file. The steps are as follow:

- Open (or create) a text file, named "result.txt" for writing.
- Write each character of the board sequentially, inserting a newline character after every 15 characters to simulate the board's 15x15 structure visually.
- After the complete board is written, write the final result message:
  - "Player 1 wins!" if Player 1 wins.
  - "Player 2 wins!" if Player 2 wins.



– "Tie" if none of the players win.

The file is then properly closed to ensure data integrity.

8 Conclusion

This project successfully implemented a playable "Five-in-a-Row" game in MIPS assembly. The challenges we faced while implementing were not limited to:

- Parsing user input manually
- Implementing a flexible and reliable win-checking algorithm
- Handling file I/O operations at the byte level
- Ensuring correct game flow even with invalid moves.

Overcoming these challenges requires one to take a careful look at low-level memory management, pointer arithmetic, and logical control flow design.

This project has helped us deepened our understanding of assembly-level programming and low-level algorithmic thinking.

Future improvements and features may include:

- Adding an AI opponent
- Allowing dynamic board sizes
- Implementing move history and undo functionality.

The source code can be found here (or in the Reference section).

References

[1] Nguyễn Minh Khôi. *Five in a Row in Assembly*, 2025. https://github.com/NguyenM1nhKho1/Five-in-a-row-In-Assembly.

[2] Wikipedia. *Gomoku*. https://en.wikipedia.org/wiki/Gomoku.